

'&=11

Beyond Real Numbers: A Hilbert-Space Exchange Format for the UniFormal (UPL) Framework

Leonardo Pedro

February 8, 2026

Abstract

The transition from structural frameworks to the computational "UniFormal" Proof Language (UPL) requires a data format that unifies Symbolic Logic with Probabilistic Physics. We propose a "Metric Exchange Format" where data is represented as vectors in a Separable Hilbert Space. In this architecture, **Dhall** expressions (canonical, normalizing) serve as the indices of the basis vectors, providing a rigorous definition of "Structure." The amplitudes are governed by **Mehler's (1866) spectral methods**, providing a rigorous definition of "Uncertainty." UPL acts as the **Turing-complete host language** that manipulates these vectors. Unlike traditional total logics, UPL supports general recursion, 'while' loops, and mutable state, allowing for the computational handling of uncertainty-aware formalizations without the artifacts of floating-point arithmetic.

1 Introduction: The "Real" Bottleneck in UniFormal

The KWARC group's evolution toward the ****UniFormal (UPL)**** initiative aims to treat *Computation* and *Deduction* as first-class citizens. However, a semantic gap remains between the discrete, total world of Logic and the continuous, often undecidable world of Physics.

Standard approaches rely on Real Numbers (approximated as floats) for physics, which strips away uncertainty information. We propose upgrading the fundamental data type to ****Vectors in a Separable Hilbert Space****.

Crucially, we distinguish between the **Data Format** and the **Language**:

- **The Data Format (The Vector - Dhall):** A static, rigorous mapping of Logical States to Probabilistic Amplitudes. This layer is ****total**** and strongly normalizing, ensuring unique canonical forms.

- **The Language (The Simulator - UPL):** The ****Turing-complete environment**** used to define, manipulate, and simulate these vectors. As seen in `test/examples/basics.p`, UPL supports imperative features like `while` loops, mutable `var`, and general recursion. This allows for powerful simulations but requires external validity predicates (the "Decidable Dense Subset") to ensure the computed objects represent valid mathematical structures.

2 Theoretical Foundations

A vector in a Hilbert space requires two things: a definition of its **Basis** (the dimensions) and a definition of its **Measure** (the length/probability).

2.1 The Symbolic Basis: Dhall Canonical Forms

For the vector space to represent logical or structural states, the basis indices cannot be arbitrary integers. They must represent unique, identifiable propositions or configurations.

We utilize ****Dhall****, a programmable configuration language that is strongly normalizing and non-Turing complete.

- **Canonicalization:** Every valid Dhall expression reduces to a unique "Normal Form." Two different ways of writing the same configuration (e.g., $2 + 2$ vs 4) resolve to the exact same hash.
- **The Mapping:** We map these unique canonical forms to the indices of our Hilbert Space basis $|e_i\rangle$.
- **Implication:** If a UPL script generates a logical statement, we normalize it (via Dhall rules) to find its "address" in the vector. This ensures that structurally identical claims share the same probability amplitude.

2.2 The Metric Measure: Mehler (1866)

Once the basis is defined by Dhall, we need a way to assign values (amplitudes) to these states in a way that allows for Field Theory (continuous variables).

We utilize Mehler's result, which proves that the limit of a uniform measure over an infinite-dimensional hypersphere converges to the Gaussian measure. This allows us to encode "Continuous Fields" as probability distributions over our discrete Dhall basis.

3 The Architecture: UPL, Dhall, and Hilbert

This proposal positions UPL not just as a proof assistant, but as a "Quantum-Like" Simulator for Logic.

3.1 The Vector as the Exchange Format

The fundamental object passed between systems is the state $|\Psi\rangle$:

$$|\Psi\rangle = \sum_i c_i \cdot |\text{Dhall}_i\rangle \quad (1)$$

- $|\text{Dhall}_i\rangle$: The **Canonical Content**. (e.g., a specific configuration of a physical field, or a logical proof step).
- c_i : The **Amplitude**. (The complex number representing the probability/weight of that configuration, governed by Mehler's measure).

3.2 UPL as the Manipulator

While Dhall is intentionally restricted (non-Turing complete) to ensure unique indexing, UPL is the **Turing-complete** language that manages the system.

- **Theories as Types/Interfaces:** As seen in 'test/examples/AI.p', Theories (like 'SearchProblem') define the *structure* or *schema* of the problem.
- **Functions as Programs:** As seen in 'test/examples/basics.p', the implementation utilizes 'while' loops, mutable state ('var'), and general recursion.

UPL defines the vector space and the operators, calculates the evolution of the vector through potentially non-terminating processes, and handles I/O. The "proof" is that the computed program (the UPL script) satisfies the type signature defined by the Theory, even if the program itself is not proven to terminate by the language's kernel.

4 Implementation Strategy

The following UPL implementation demonstrates this architecture. We explicitly construct the Complex Hilbert Space $\mathcal{H}$ starting from Complex Rational Numbers.

Note on Totality: Since UPL is Turing-complete, the functions defined below (like 'vec_index') are programs that *compute* values. Their validity as mathematical objects (i.e., that they terminate and converge) is not enforced by a totality checker but is ensured by adhering to the "Decidable Dense Subset" strategy. The Complex Rationals (or finite Dhall ASTs) form the computable skeleton of the complete space.

```
theory HilbertSpace {    include Numbers.Complex
```

Our fundamental object is a **Double-Indexed Sequence** $s(k, n)$, representing the n -th complex rational approximation of the k -th coordinate of a vector.

```
type h_vector = (int, int) -> num    row = (s, n) -> k -> s(k, n)
```

Index Function for Vector Coordinates To comply with L_{ω_1, ω_1} logic, we avoid passing infinite-dimensional objects as function arguments. Instead, we define an index function that extracts a single coordinate (which is itself a Cauchy sequence of complex rationals) from a vector:

```
vec_index : (h_vector, int) -> complex    vec_index = (v, k) -> n -> v(k, n)
```

This function takes a vector ‘v’ and a coordinate index ‘k’, returning the ‘k’-th coordinate as a ‘complex’ number (a Cauchy sequence). This allows functions to work with a finite number of arguments, where each argument represents an infinite-dimensional object accessed through the index.

For convenience, we also define a low-level index for extracting individual approximations:

```
elem_at = (v: h_vector, k: int, n: int) -> num    elem_at = (v, k, n) -> v(k, n)
```

4.1 The Validity Predicates

To ensure we are selecting valid members of the completion (and not just random noise), we impose two conditions corresponding to the logic’s requirements:

1. Square Integrability (The Skeleton) Each approximation row must be a finite-norm vector in the underlying complex rational space. The norm is computed using the complex inner product.

Rather than passing entire rows as arguments, we use the row accessor and work with coordinate indices:

```
is_l2_at_level : (h_vector, int) -> type    c_inner_at : (h_vector, h_vector, int) -> type
c_inner_at = (v1, v2, n) -> sum_over_k(times(conj(v1(k,n)), v2(k,n)))
c_sq_norm_at = (v: h_vector, n: int) -> c_inner_at(v, v, n)
```

2. Cauchy Convergence (The Limit) The sequence of rows must converge. Using the index function, we express convergence at the coordinate level:

```
coord_cauchy = (v: h_vector, k: int) -> type
forall eps. leq(z, eps) & !(eps == z) =>
  exists N. forall n,m.
    leq(N, n) & leq(N, m) =>
      c_modulus(minus(v(k,n), v(k,m))) < eps
valid_h_vector = v -> (forall n. is_l2_at_level(v, n)) &
  (forall k. coord_cauchy(v, k))
```

4.2 Emergence of Complex Scalars

Traditional approaches postulate Complex Numbers ($\mathbb{C}$) as a black box. In this constructive view, $\mathbb{C}$ *emerges* naturally from the geometry of $\mathcal{H}$.

The inner product of two vectors is computed using the index function to access coordinates. Rather than passing infinite-dimensional rows, we use coordinate-level access:

```
h_inner_at = (v1, v2, n: int) -> c_inner_at(v1, v2, n)    h_inner = (v1, v2) -> n -> c_inner_at(v1, v2, n)
```

Thus, we define the Complex numbers simply as the scalar field of our completion:

```
type complex = int -> num
```

We must also define when two Cauchy sequences represent the same Complex Number. They are equivalent if their difference converges to zero. For complex numbers, we measure convergence using the modulus (absolute value).

```
c_modulus = (c: num) -> sqrt(times(c, conj(c)))    complex_zero = (c: complex) -> for
exists N. forall n. leq(N,n) => c_modulus(c(n)) < eps    complex_diff = (c1, c2) ->
complex_equiv = (c1, c2) -> complex_zero(complex_diff(c1, c2))
```

4.3 Constructive Dimension

We define the dimension of our space constructively using the concept of **Finite Support**.

A vector has finite support if its components eventually become (equivalent to) zero. This corresponds to the **Decidable Dense Subset** (the set of finite linear combinations of basis vectors).

Using the index function, we extract coordinates without passing infinite-dimensional objects:

```
has_finite_support = (v: h_vector) ->
exists K. forall k. leq(K, k) => complex_zero(vec_index(v, k))
basis_vec = i -> (k, n) -> (if (k == i) o else z)    dense_subset_axiom : |- forall s. h
```

4.4 Transcendental Functions

Having constructed the Complex numbers as Cauchy sequences of Complex Rationals, we can now define transcendental functions (exponential, sine, cosine) via their power series expansions. These are fundamental for analyzing operators on Hilbert spaces.

Factorial and Power First, we need factorial for series coefficients and exponentiation:

```
factorial = (n: int) -> if (leq(n, z)) o else times(n, factorial(p(n)))
power = (x: num, n: int) -> if (leq(n, z)) o else times(x, power(x, p(n)))
```

Complex Exponential The complex exponential is defined via its power series: $\exp(z) = \sum_{n=0}^{\infty} \frac{z^n}{n!}$

For a complex number c (a Cauchy sequence of complex rationals), we define $\exp(c)$ as the Cauchy sequence where each approximation is the partial sum of the series.

```
exp_partial = (x: num, N: int) ->
if (leq(N, z)) o else plus(frac(power(x, N), factorial(N)), exp_partial(x, p(N)))
exp = (c: complex) -> n -> exp_partial(c(n), n)
Key properties:    exp_zero : |- complex_equiv(exp(n -> z), n -> o)
exp_add : |- forall c1, c2. complex_equiv(exp(complex_sum(c1, c2)), complex_prod(exp(c1), exp(c2)))
where we define complex arithmetic:    complex_sum = (c1, c2) -> n -> plus(c1(n), c2(n))
complex_prod = (c1, c2) -> n -> times(c1(n), c2(n))    complex_conj = (c: complex) -> n -> conj(c(n))
complex_neg = (c: complex) -> n -> uminus(c(n))
```

Sine and Cosine via Euler's Formula Euler's formula states: $e^{ix} = \cos(x) + i \sin(x)$

We can extract sine and cosine from the complex exponential: $-\sin(x) = \frac{e^{ix} - e^{-ix}}{2i}$ - $\cos(x) = \frac{e^{ix} + e^{-ix}}{2}$

In UPL, using the imaginary unit $i = \text{comp}(z, o)$:

```
i_unit = comp(z, o)    i_times_complex = (c: complex) -> n -> times(i_unit, c(n))
sin = (c: complex) -> n ->
  frac(minus(exp(i_times_complex(c))(n), exp(i_times_complex(complex_neg(c)))(n)),
        times(comp(z, s(s(z))), o))    cos = (c: complex) -> n ->
  frac(plus(exp(i_times_complex(c))(n), exp(i_times_complex(complex_neg(c)))(n)),
        times(s(s(z)), o))
```

The Pythagorean identity follows from Euler's formula: pythagorean : |- forall c. complex_equality
}

5 Discussion

This architecture unifies three distinct needs in formalization:

1. **Structure (Dhall):** Ensuring that logical statements have a unique, decidable identity.
2. **Uncertainty (Mehler):** Ensuring that we can perform calculus and field theory with rigorous probability tracking.
3. **Computation (UPL):** Providing a powerful language to execute simulations using these safe data types.

This resolves the tension between "Formal Rigor" and "Numerical Utility." We do not force Physics to become Discrete Logic; instead, we use UPL to map the Continuous Physics into a Discrete, Probability-Preserving Hilbert Space indexed by Logical Structures.

5.1 Contemporary Validation: nGPT and Hypersphere Learning

The proposed data format—representing information as normalized vectors in a Hilbert space—finds contemporary validation in modern machine learning architectures. Notably, **nGPT (normalized GPT)** from NvidiaAI demonstrates that large language models can learn entirely on the hypersphere, where all representations are normalized vectors. This architecture replaces traditional dot products with cosine similarity and maintains all embeddings, weights, and activations as unit-norm vectors on a hypersphere.

The success of nGPT in language modeling tasks provides empirical evidence that normalized vector representations on hyperspheres are not merely a theoretical construct but a practical and effective data format for complex computational systems. This aligns perfectly with our proposal: just as nGPT uses the hypersphere geometry for learning language patterns, our framework

uses Hilbert space geometry to unify logical structures (indexed by Dhall) with probabilistic amplitudes (governed by Mehler’s measure). The key insight is that **normalization** (maintaining unit vectors) provides both numerical stability and a natural probabilistic interpretation—properties essential for both machine learning and formal reasoning under uncertainty.

A Historical Foundations and Detailed Logic

For completeness, we include the comprehensive theoretical context regarding infinitary logic, set theoretic paradoxes, and the mathematical foundations that motivate this framework.

A.1 The Challenge of the Infinite: Cantor and Skolem

Cantor’s Theorem (see also **Skolem’s Paradox**) demonstrates that the power set of an infinite set has a strictly greater cardinality than the set itself. In the context of logic, this creates a fundamental tension between **semantics** (the set of models/truths, analogous to the power set) and **syntax** (the set of proofs/sentences, which remains countable).

Ideally, logic should be both **expressively complete** (able to uniquely describe structures like the Real numbers or Hilbert spaces) and **deductively complete** (where every semantic truth has a syntactic proof). However, we are generally forced to choose. Standard First-Order Logic (FOL) is deductively complete but expressively weak; it cannot distinguish between the standard models (e.g., $\mathbb{N}, \mathbb{R}$) and non-standard models containing hyper-elements (e.g., the Hyperreals). This inability to pin down the unique standard model represents a fundamental “knowledge gap”—an objective ambiguity in our formalism.

There are many ways of generalizing First-Order Logic to close this gap, but as argued by Goodstein (see www.psiquadrat.de/downloads/goodstein63.pdf), the knowledge gap is unavoidable with countable proofs. What is needed is a framework to formalize this lack of information.

A.2 Probability as Political Choice

We can manage this uncertainty if we choose what is relevant to us. This choice is effectively **political**: since there is no objective “degree of ignorance” (ignorance being the absence of information), we must impose a structure on the unknown based on relevance. **Probability** is the objective representation of these political choices. By assigning a probability measure, we fill the logical gaps with a weighted preference, transitioning from absolute truths (which may be undecidable) to likely outcomes (which are calculable).

A.3 Mehler’s 1866 Discovery

In 1866, Mehler (see opencommons.uconn.edu/dissertations/2137 for an updated version of his work) defined a large class of real functions with a countably

infinite number of real variables in hyperspherical coordinates. While Cantor’s Theorem implies that the set of *all* such functions has the cardinality of the power set of the continuum ($2^{\mathfrak{c}}$), Mehler’s formalism allows this space to be accessed and approximately parameterized using only the cardinality of the continuum ($\mathfrak{c}$), by defining a probability measure over the space of functions.

Mehler demonstrated that Hermite polynomials arise as the limit of Gegenbauer polynomials on an infinite-dimensional sphere, effectively defining a probability measure over a space of wave-functions. Mathematically, Mehler established a simple non-perturbative Quantum Field Theory (for all practical purposes), more than 30 years before the term “quantum” entered the physical lexicon through Planck. He arrived at these probabilistic tools—the mathematical backbone of the Standard Model of Particle Physics—as an unintended byproduct of solving the geometry of infinite variables. Mehler never used the terms “probability” or “physics” in his article; rather, he was interested in how to define a function of infinite variables in hyperspherical coordinates.

A.4 Computation and the Decidable Dense Subset

Arbitrarily large computers can only verify a countable number of proofs, due to our physical limits irrespectively of the details of Mathematical theory involved. Note that in some mathematical theories, we can prove (and verify) that the proofs of an uncountable number of statements exist, despite that we cannot verify them one by one, but such meta-proof is done in a meta-theory of Mathematics, so that meta-theory is the mathematical theory we refer to in the following.

We map one-to-one each proof we choose to verify to an element of a basis of a separable Hilbert space; this is valid in principle for all (provable) Mathematics. Fortunately, separable Hilbert spaces contain a **decidable dense subset**—specifically, the set of finite linear combinations of basis vectors with rational coefficients.

The full Hilbert space also contains vectors defined by undecidable processes (where the generating algorithm may never halt). For such a vector, does a computable approximation exist? The density of the subset guarantees that a rational approximation exists arbitrarily close to the vector. But we cannot algorithmically *identify* which computable element is the approximation because we cannot compute the target. Since a computable approximation exists but remains unknown, then we must guess by using a probability measure, provided the underlying space is complete.

A.5 The Hybrid Logical Framework

This necessitates a hybrid logical framework:

1. **The Ontological Foundation (L_{ω_1, ω_1}):** To ensure our probability measures have a valid domain, we must refer to the standard, separable,

complete Hilbert space. We need the expressive power of L_{ω_1, ω_1} (allowing infinite quantifiers) to rigorously define this unique structure. This guarantees that the “holes” between our rational approximations are filled (metric completion), as strictly weaker logics cannot distinguish the “real” topological space from its incomplete rational skeleton.

2. **The Epistemological Engine (Countable Fragments of $L_{\omega_1, \omega}$):** While L_{ω_1, ω_1} guarantees the space exists, it is logically incomplete. Therefore, to perform actual derivations, we move to $L_{\omega_1, \omega}$. However, a subtle restriction is required: although the vocabulary is countable, the full set of sentences in $L_{\omega_1, \omega}$ is uncountable. Since computation is limited to countable steps, we cannot assign probabilities to every formula in the full logic. Instead, we work within a **Countable Fragment** of $L_{\omega_1, \omega}$ —a recursively definable subset of sentences grounded in our decidable dense subset. This fragment is **complete** (in the sense of proofs) and allows us to prove the existence of approximations from the computable skeleton that converge to our chosen probability distribution.

Optionally, the “computable skeleton” can be implemented in **non-Turing complete languages** (such as Dhall) using total functional programming. This ensures that every approximation step is decidable (halts in finite time), where each statement has a canonical form identified by an integer.

A.6 Countable Additivity Caveat

A critical technical caveat arises regarding **countable additivity**. Strict adherence to a countable fragment prevents us from strictly proving statements that rely on the uncountable union of null sets—a standard requirement in full measure theory. The solution lies in a shift of perspective: we treat the probability measure not as a function over *all* possible subsets of the space, but as a consistent assignment of values to the **formulas** within our countable fragment. By restricting our quantification to these **definable sets** (essentially the Borel sets generated by the fragment), we bypass the need to manage the full power set. This approach aligns perfectly with **Computable Measure Theory**, ensuring that our probabilistic reasoning remains effectively calculable.

A.7 Kopperman’s Contribution

Thus, we rely on the stronger, incomplete logic to guarantee the existence of the continuum (the territory), while using a countable fragment of the weaker, complete logic to construct the proofs via a decidable dense subset (the map). **Ralph Kopperman**, in his 1967 paper “*The L_{ω_1, ω_1} -Theory of Hilbert Spaces*”, provided the ingredients necessary to formalize all we discussed above.

A.7.1 What is $L_{\omega_1 \omega_1}$?

Standard logic is $L_{\omega \omega}$: finite quantifier depth, finite formulas. $L_{\omega_1 \omega_1}$ allows **countable conjunctions** ($\bigwedge_{i=0}^{\infty} \phi_i$) and **countable quantifiers**. This expres-

sive power lets us pin down properties like "for all integers $n \dots$ " or "there exists a limit...".

A.7.2 Formal Definition: $L_{\pi\epsilon}$

Kopperman generalizes classical logic to languages $L_{\pi\epsilon}^t$, characterized by two cardinal bounds:

1. π : The bound on conjunctions and disjunctions.
2. ϵ : The bound on quantifier strings.

To capture the **Ontological Foundation** of Hilbert Spaces, we require $L_{\omega_1\omega_1}$:

- $\pi = \omega_1$: We can form conjunctions of countable sets of formulas ($\bigwedge_{i < \omega} \phi_i$).
- $\epsilon = \omega_1$: We can quantify over countable sets of variables ($\forall \{x_i\}_{i < \omega} \dots$).

This logical strength is necessary for the **Completeness Axiom**. Standard First-Order Logic ($L_{\omega\omega}$) cannot distinguish between the standard Reals and "gapped" fields like the Rationals because it cannot express "for **every** sequence...". $L_{\omega_1\omega_1}$ allows us to quantify over entire sequences, effectively locking down the unique complete structure.

In our UPL implementation, we model these "infinite" quantifiers via **Dependent Types**, quantifying over function types such as `int -> univ` (sequences).

A.8 The Countable Fragment and UPL

While $L_{\omega_1\omega_1}$ ensures the model exists, our **Epistemological Engine**—the actual proofs we construct in UPL—operates within a **Countable Fragment**.

We do not use every possible sentence in $L_{\omega_1\omega_1}$. Instead, we restrict ourselves to recursively definable sentences that describe operations on our **Decidable Dense Subset** (the computable skeleton). In UPL, this corresponds to:

- **The Skeleton**: The Rational Numbers (`Numbers.Rat`) and finite lists of them.
- **The Limit**: The concept of a Cauchy Sequence, defined via a predicate.

A.9 Representation Theorems

Kopperman proves that our axiomatic definitions fully capture the intended mathematical objects:

1. **Reflexive Real System**: Any structure satisfying the axioms is isomorphic to $\mathbb{R}$.
2. **Hilbert System**: Any structure satisfying the axioms corresponds uniquely to a **Hilbert Space**.

B Appendix B: Formalization of Hamiltonian Operators

This appendix provides a UPL formalization of key results from mathematical physics, demonstrating how the "Theories as Types" architecture captures advanced operator theory.

B.1 Basic Operator Theory

We define operators on our Hilbert Space $\mathcal{H}$. An operator serves as a schema for valid transformations.

```
theory OperatorTheory {
  include HilbertSpace      type Operator = h_vector -> h_vector
  include Logic.Predicates  // Domain is a predicate on vectors identifying the subspace
  type Domain = h_vector -> prop // Basic properties
  Hermitian : Operator -> prop
  SelfAdjoint : Operator -> prop
  Positive : Operator -> prop // D is a core for N
  Core : (Operator, Domain) -> prop
  Included : (Domain, Domain) -> prop
  // Commutator: [A, B] = AB - BA
  Commutator = (A: Operator, B: Operator) -> phi -> min
  norm : h_vector -> num }

```

B.2 Faris-Lavine Corollary 1.1

The following corollary from Faris and Lavine (1974) provides a condition for essential self-adjointness using commutators.

```
theory FarisLavine {
  include OperatorTheory      EssentialSelfAdjoint : (Operator, Domain) -> prop
  corollary_1_1 : |-          forall H, N : Operator.      forall D : Domain.
    // Assumptions on Geometry
    Hermitian(H) & Positive(N) & SelfAdjoint(N) &
    Core(N, D) & Included(D, Domain(H)) & // (i) H is bounded relative to N
    (exists a, b. forall phi. D(phi) => norm(H(phi)) <= a * norm(N(phi)) + b * norm(phi))
    // (ii) Commutator is bounded relative to N
    (exists c, d. forall phi. D(phi) => norm([H, N](phi)) <= c * norm(N(phi)) + d * norm(phi))
    // Conclusion
    => EssentialSelfAdjoint(H, D) }

```

B.3 Friedrichs Canonical Extension

The Friedrichs extension constructs a self-adjoint extension of any semi-bounded symmetric operator.

```
theory FriedrichsExtension {
  include OperatorTheory      Symmetric : Operator -> prop
  Graph : Operator -> (h_vector, h_vector) -> prop
  Graph = op -> (x, y) -> y == op(x)
  // A sesquilinear form Q associated with S
  type Form = (h_vector, h_vector) -> complex
  AssociatedForm = (S: Operator) -> (u, v) -> h_inner(S(u), v)
  PreservesLowerBound : (Operator, Form) -> prop
  friedrichs_extension : |-
    forall S : Operator.      Positive(S) & Symmetric(S) =>
    exists H : Operator.      SelfAdjoint(H) & // H extends S
    Included(Graph(S), Graph(H)) & // The extension preserves the lower bound of S
    PreservesLowerBound(S, H) }

```

References

1. **Mehler, F. G.** (1866). *Ueber die Entwicklung einer Function von beliebig vielen Variablen nach Laplaceschen Functionen höherer Ordnung*. J. Reine Angew. Math, 66, 161-176.
2. **Kopperman, R.** (1967). *The $L_{\omega_1\omega_1}$ -Theory of Hilbert Spaces*. Journal of Symbolic Logic.
3. **Gabriel Gonzalez.** *The Dhall Configuration Language*. <https://dhall-lang.org>
4. **Rabe, F. et al.** (UniFormal / KWARC). *The MMT Language and System*.
5. **Ben Yaacov, I., et al.** (2008). *Model theory for metric structures*.
6. **Tanaka, K.** (2019). *Shift-invert Rational Krylov method*.
7. **Doerschuk, P. et al.** (2019). *Evaluation of the discretization error in the computation of the eigenvalues of the fractional Laplacian*. Springer.
8. **Loshchilov, I., Hsieh, C., Sun, S., & Ginsburg, B.** (2024). *nGPT: Normalized Transformer with Representation Learning on the Hypersphere*. NvidiaAI. <https://github.com/lucidrains/nGPT-pytorch>